COMP3821/9801

Workshop Week 4

Matroids and Minimum Spanning Trees

Copyright Disclaimer. *Reproducing, publishing, posting, distributing, translating this document is an infringement of copyright and will be referred to UNSW Student Conduct and Integrity for action.*

Guided Problem

The demonstrators will solve the following problems. For the rest of the workshop, work in small groups to solve the remaining workshop exercises. There are optional problems at the end of this document for further practice.

Guided Problem

Let $k_1, \ldots, k_\ell$ be non-negative integers. Let E be a finite ground set that is partitioned into sets $E_1, \ldots, E_\ell$. In other words, we divide E into ℓ subsets such that

$$E_i \cap E_j = \emptyset \quad \forall i \neq j, \quad \bigcup_i E_i = E.$$

Consider the following family of sets

$$\mathcal{I} = \{X \subseteq E : |X \cap E_i| \leq k_i,\ \forall i \in \{1, \ldots, \ell\}\}.$$

(a) Prove that $(E, \mathcal{I})$ forms a matroid.

(b) Let $E = \{e_1, \ldots, e_n\}$ be a ground set on n elements and suppose that each element $e_i \in E$ has a non-negative weight $w_i \geq 0$. Let $\mathcal{E} = \{E_1, \ldots, E_\ell\}$ be a partition of E; to visualise the partition, we can assign each element e_i one of ℓ colours. We are also given integers $k_1, \ldots, k_\ell$. We say that $S \subseteq E$ is *feasible* if it contains no more than k_i elements from E_i for each $i \in \{1, \ldots, \ell\}$.

To keep things simple, assume you have access to an oracle $\textsc{Feasible}(S)$ with the following description and each oracle call takes constant time.

- **Input.** A subset $S \subseteq E$ of elements.
- **Output.** Returns whether it is feasible set.

Describe and analyse a polynomial-time algorithm that returns a subset $S \subseteq E$ of k elements such that

- no more than k_i elements is chosen from E_i, for each $1 \leq i \leq \ell$, and
- $\sum_{i \in S} w_i$ is maximised.

Guided Problem

Let $G = (V, E)$ be a connected, weighted, and undirected graph, where each edge has non-negative weight. Let $S \subseteq V$ be a subset of vertices. Describe and analyse a polynomial-time algorithm to compute a minimum-weighted forest such that, for each vertex $v \in V \setminus S$, there is a path between v and *exactly* one vertex in S.

Workshop Exercises

1. Let $G = (V, E)$ be a connected, undirected, and weighted graph where all of the weights are non-negative. Let $I \subseteq E$ be a subset of edges. We say that I is *independent* if deleting I from G still leaves the graph connected. Let $\mathcal{J}$ denote

the set of all independent sets. For this problem, you may assume that $(E, \mathcal{J})$ forms a matroid.

(a) Consider the problem of choosing a maximum weighted subset $X \subseteq E$ of edges to delete such that $G - X$ is still connected. Show that

$$\mathsf{MST}(G) = \sum_{e \in E} w(e) - w(X),$$

where $w(X)$ denotes the sum of the weight of the edges in X.

(b) Hence or otherwise, describe and analyse a polynomial-time algorithm that computes the maximum weighted set of edges whose deletion still leaves G connected.

2. We consider a scheduling problem with deadlines and penalties. We are given a set S of n tasks. Each task takes 1 unit of time to complete. Task i has a deadline $1 \leq d_i \leq n$ and we can freely choose to complete the task any time before its deadline. However, if we do not complete the task by its deadline, then we incur a non-negative penalty $p_i \geq 0$. Furthermore, we can only process one task at a time. Finally, a *schedule* for S is a permutation of S that specifies the order in which to perform the tasks.

(a) Let $A \subseteq S$ be a subset of tasks. We say that A is *independent* if there exists a schedule that can process all tasks in A such that no task is completed after its deadline. For each $t \in \{0, \ldots, n\}$, let $N_t(A)$ denote the number of tasks in A whose deadline is t or earlier. Show that A is independent if and only if $N_t(A) \leq t$ for all $t \in \{0, 1, \ldots, n\}$.

(b) Let $\mathcal{J}$ be the set of all independent sets. Show that $(S, \mathcal{J})$ forms a matroid.

(c) Hence or otherwise, describe and analyse a polynomial-time algorithm that computes a schedule for S that minimises the total penalty incurred for missed deadlines.

To keep things simple, assume you have access to an oracle $\textsc{Feasible}(A)$ with the following description and each oracle call takes constant time.

- **Input.** A subset $A \subseteq S$ of tasks.
- **Output.** Returns whether one can schedule the tasks in A so that no task is completed after its deadline.

3. Let $G = (V, E)$ be an undirected and weighted graph where each edge $e \in E$ has weight $w(e)$. Note that the weight may be positive, negative, or zero. A *feedback edge set* is a subset $F \subseteq E$ of edges such that every cycle in G contains at least one edge in F; that is, if we remove F from the graph, then the resulting graph becomes acyclic. Describe and analyse a polynomial-time algorithm that computes the minimum-weight feedback edge set of G.

Note. *If the graph is **directed**, then the problem becomes* NP*-hard.*

Optional Exercises

These exercises will not contain solutions. We encourage that you post on the forums to discuss the approaches for these problems. Additionally, you may find the hints helpful.

4. Let M be a matroid. Prove that all maximal independent sets have the same cardinality.

   Note. *We call this the **rank** of the matroid.*

5. Let $M = (S, \mathcal{J})$ be a weighted matroid, where each element $e \in S$ has an associated non-negative weight $w(e) \geq 0$. Assume that the elements of S are ordered in non-increasing order by weight and let x be the first element of S such that $\{x\}$ is independent. Prove that, if x exists, then there exists an optimal subset $A \subseteq S$ that contains x.

   Note. *This implies that matroids exhibit the greedy-choice property.*

6. Let $M = (S, \mathcal{J})$ be a weighted matroid, where each element $e \in S$ has an associated non-negative weight $w(e) \geq 0$. Let x be the first element chosen by the greedy matroid algorithm. Define the set system $M' = (S', \mathcal{J}')$, where

   - $S' = \{y \in S : \{x, y\} \in \mathcal{J}\}$,
   - $\mathcal{J}' = \{B \subseteq S \setminus \{x\} : B \cup \{x\} \in \mathcal{J}\}$.

   We prove that matroids exhibit the optimal substructure property.

   (a) Prove that M' forms a matroid.

   (b) Prove that a maximum weight solution in M that contains x corresponds to a maximum weight solution in M'; that is, if $A \subseteq S$ is a maximum weight independent set that contains x, then $A' := A \setminus \{x\}$ is a maximum weight independent set of M'.

   Note. *Combining the previous problem with this problem yields the correctness proof for the greedy algorithm on weighted matroids.*

7. Let $M_k = (S, \mathcal{J}_k)$ be a set system, where $\mathcal{J}_k$ denotes the collection of all subsets of S with size at most k, with $0 \leq k \leq |S|$. Prove that M_k is a matroid.

   Note. *This is known as the **uniform matroid of rank** k.*

8. Let $M = (S, \mathcal{J})$ be a matroid. Let $\mathcal{B}$ be the set of bases of M. Then, for all $X, Y \in \mathcal{B}$, $x \in X \setminus Y$ prove that there exists some $y \in Y \setminus X$ such that $(X \setminus \{x\}) \cup \{y\} \in \mathcal{B}$.

   Note. *This is known as Steinitz' exchange lemma.*

9. Let $M = (S, \mathcal{J})$ be a matroid. Let $M^* = (S, \mathcal{J}^*)$ be a set system, where

$$\mathcal{J}^* = \{A' : S \setminus A' \text{ contains some maximal independent set } A \in \mathcal{J}\}.$$

   (a) Prove that M^* is a matroid.

   (b) Hence, conclude that the matroid from Problem 1 is, indeed, a matroid.

   Note. *This is known as the **dual matroid** and the matroid from Problem 1 is the **co-graphic matroid**. Note that the maximal independent sets of M are complements of the maximal independent sets of M^*. Furthermore, it is not too difficult to see that $(M^*)^* = M$.*

10. Let $G = (V, E)$ be an undirected graph. We say that a subset $V' \subseteq V$ of vertices is *matched* if there exists a matching $M \subseteq E$ such that every vertex in V' is incident to some edge in M. Let $\mathcal{J}$ denote the set of all matched subsets of vertices. Prove that $M = (V, \mathcal{J})$ forms a matroid.

    Note. *This is called the **matching matroid**.*

11. Let S be a finite ground set and let $\mathscr{A}$ be a family of subsets of S. We say that $\mathscr{A}$ is *laminar*, if for all $A_1, A_2 \in \mathscr{A}$, either $A_1 \cap A_2 = \emptyset$, or $A_1 \subseteq A_2$ or $A_2 \subseteq A_1$; in other words, for any pair of sets in $\mathscr{A}$, either they are disjoint, or one is contained in the other. Let $\mathscr{A}$ be a laminar family and assign a weight function $w : \mathscr{A} \to \mathbb{R}_{\geq 0}$. Let $I \subseteq S$. We say that I is *independent*, if for all $A \in \mathscr{A}$, we have that

    $$|I \cap A| \leq w(A).$$

    Let $\mathcal{J}$ denote the set of all independent sets. Show that the set system $M = (S, \mathcal{J})$ forms a matroid.

    Note. *This is called a **laminar matroid**.*

12. Let $G = (V, E)$ be an undirected graph. A subset $E' \subseteq E$ of edges is said to be *independent* such that each connected component contains at most one cycle. Let $\mathcal{J}$ denote the collection of all independent sets. Prove that $M = (E, \mathcal{J})$ forms a matroid.

    Note. *This is called the **bicircular matroid**.*

13. Let $G = (V, E)$ be a directed graph, and let $s \in V$ be a vertex. A subset $V' \subseteq V$ of vertices is *independent* if there are edge-disjoint paths from s to every vertex in V'. Let $\mathcal{J}$ denote the collection of independent sets. Prove that $(V, \mathcal{J})$ forms a matroid.

14. Let $M = (S, \mathcal{J})$ be a matroid and let $w : S \to \mathbb{R}$ be its weight function.

    (a) Show that if w is injective, then there is a unique basis of maximum weight. In other words, there is a unique optimal solution.

    (b) Show that if w is not injective, then the greedy algorithm can return an arbitrary basis of maximum weight.

15. Let $G = (V, E)$ be an undirected graph, and let $C = \{C_1, \ldots, C_k\}$ be a collection of cycles. We say that C is *useless* if every edge in E appears in an even number of cycles in C. Finally, we say that C is *independent* if C contains no useless subset. A maximal independent set of cycles is called a *cycle basis* for G.

    (a) Let C be a cycle basis for G. Prove that for any cycle c in G, there is a subset $A \subseteq C$ such that $A \cap \{c\}$ is useless.

    (b) Let $\mathcal{J}$ denote the collection of independent sets. Prove that $(E, \mathcal{J})$ forms a matroid.

    (c) Suppose that each edge of G has a weight. Define the weight of a cycle to be the total weight of its edges, and the weight of a *set* of cycles to be the total weight of all cycles in the set. In other words, each edge is counted once in every cycle that it appears in; it could appear in multiple cycles.

    Describe and analyse a polynomial-time algorithm to compute the minimum-weight cycle basis of G.

16. There are n vendors, partitioned into zones $Z_1, \ldots, Z_p$. Choosing vendor i yields profit $w(i) \geq 0$, which is a non-negative integer. Inside each zone Z_i, the vendors are partitioned into m different types of stalls $T_{i,1}, \ldots, T_{i,m}$. You want to choose a set S of vendors such that, for each zone Z_i,

    - at most one vendor of each type $T_{i,j}$ is chosen,
    - at most $b(i)$ vendors are chosen from zone Z_i,

    where we are given a "budget" function $b : \{1, \ldots, p\} \to \mathbb{Z}$ that tells us how many vendors we can take for each zone. In addition, we want to choose at most k vendors in total. Finally, the profit of the set S of chosen vendors is equal to

    $$\mathsf{profit}(S) = \sum_{i \in S} w(i).$$

    Describe and analyse a polynomial-time algorithm that chooses a set S of at most k vendors that maximises the total profit.

17. Let $G = (V, E)$ be a connected, undirected, and weighted graph. Furthermore, suppose that all of the weights of G are distinct. Prove that there is a unique minimum spanning tree.

18. Let $G = (V, E)$ be a connected and weighted graph; otherwise, no such spanning tree exists. We now describe a more generic minimum spanning tree algorithm. Throughout, we shall maintain an acyclic subgraph F of the graph G, which we call an *intermediate spanning forest*. Each connected component of F is a subtree and we recursively merge a pair of subtrees via an edge until we obtain one connected component, which is a spanning tree of G.

    Let C be a component of F. We call an edge *safe* if it is the minimum-weighted edge with exactly one end point in C. Furthermore, call an edge *useless* if it was not added to F and it has endpoints in the same component.

    (a) Prove that the minimum spanning tree contains no useless edge.

    (b) Prove that the minimum spanning tree contains every safe edge.

19. We consider the following algorithms.

    (a) The forest F contains only one nontrivial component T; every other component is an isolated vertex. Repeatedly add all of the safe edges of T to the component T.

    (b) Scan the edges in increasing order of weight. If an edge is safe, then add it to F.

    (c) Recursively add all of the safe edges.

    Prove that each of these algorithms produce a minimum spanning tree; hence, describe and analyse a polynomial-time algorithm for each of these algorithms.

20. Let $G = (V, E)$ be a connected, weighted, and undirected graph. Furthermore, suppose that weights are all distinct so there is a unique minimum spanning tree. A *cut* is a partition of the vertices into sets $(X, V \setminus X)$. We say that an edge e *crosses* the cut if one endpoint of e lies in X and the other lies in $V \setminus X$.

    We can reformulate the classical minimum spanning tree algorithms via the following two rules. Let T be the minimum spanning tree of G.

- **Red rule** (cycle property)**.** For all cycles C in G, the maximum weight edge in C is *not* in T.
- **Blue rule** (cut property)**.** For any cut $(X, \bar{X})$, the minimum weight edge crossing the cut is in T.

Prove that repeatedly applying the red and blue rules in any arbitrary order will produce the minimum spanning tree.

Note. *One can reformulate this on matroids.*

21.

(a) Describe and analyse a polynomial-time algorithm that finds the second smallest spanning tree; that is, you want to design an algorithm that computes the spanning tree with the smallest weight that is not the minimum spanning tree.

(b) Given an integer $k \geq 1$, generalise the above algorithm to find the k spanning trees of G with smallest weight.

22. Let $G = (V, E)$ be a weighted *planar* graph. Describe and analyse an $O(|V|)$ algorithm that computes the minimum spanning tree of G.

Hints

4. Use the exchange property and prove by contradiction.
5. Use a standard exchange argument.
6. Let $A' := A \setminus \{x\}$. Show that $w(A) = w(A') + w(x)$.
7. If $|X| < |Y|$, then show that there exist some element $x \in Y \setminus X$. How large can $|X \cup \{x\}|$ be?
8. Since X is a basis, you know that $X \setminus \{x\} \in \mathcal{J}$. Then use the exchange property of matroids to show that $(X \setminus \{x\}) \cup \{y\} \in \mathcal{J}$.
9. For the exchange property, you might want to use Steinitz' exchange lemma.
10. Consider the symmetric difference of two matchings.
11. Consider $X, Y \in \mathcal{J}$ such that $|X| < |Y|$, and let $e \in Y \setminus X$. The case where $e \notin A$ for any $A \in \mathscr{A}$ is easy. Therefore, suppose that, for each $e \in Y \setminus X$, there exists some $A \in \mathscr{A}$ such that $e \in \mathscr{A}$. Let $\mathscr{A}(e)$ denote the collection of sets such that $e \in A$ for each $A \in \mathscr{A}(e)$. It is easy to show that $|(X \cup \{e\}) \cap A| \leq w(A)$, for each $A \in \mathscr{A} \setminus \mathscr{A}(e)$. Show that $|(X \cup \{e\}) \cap A| \leq w(A)$ for each $A \in \mathscr{A}(e)$.
12. For the exchange property, show that if X has less elements than Y, then X has some connected component with no cycle and show that one can always connect the connected component with no cycle to another component using an edge in Y.